01. Omni 네트워크 전체 구조 분석

개요

Omni는 크로스체인 메시징과 EVM을 통합한 복합적인 블록체인 프로토콜입니다. 이 문서는 Omni 모노레포의 전체 구조를 분석하고 각 컴포넌트의 역할과 상호작용을 정리합니다.

프로젝트 구조

루트 레벨 구조

```
omni/
├── cli/ # Omni 명령줄 인터페이스
├── contracts/ # 솔리디티 스마트 컨트랙트
├── docs/ # 문서 및 웹사이트 리소스
├── e2e/ # 엔드투엔드 테스트 프레임워크
├── halo/ # Omni 합의 클라이언트
├── lib/ # 핵심 라이브러리
├── monitor/ # 네트워크 모니터링 서비스
├── octane/ # EVM용 Cosmos SDK 모듈
├── relayer/ # 크로스체인 메시지 릴레이어
├── scripts/ # 개발 및 운영 유틸리티 스크립트
├── sdk/ # Omni 애플리케이션 개발 SDK
```

```
├── solver/ # SolverNet 참조 구현
├── anchor/ # Solana Anchor 프로그램
├── go.mod # Go 모듈 정의
├── Makefile # 빌드 및 배포 자동화
└── README.md # 프로젝트 개요
```

핵심 컴포넌트 분석

1. Halo (합의 클라이언트)

위치: `halo/`

역할: Omni 네트워크의 핵심 합의 엔진

주요 기능:

- Cosmos SDK 기반 블록체인 애플리케이션
- ABCI++ 합의 플로우 구현
- EVM 엔진 통합 (`octane/evmengine`)
- 크로스체인 증명 관리 (`AttestKeeper`)
- 검증자 동기화 (`ValSyncKeeper`)

핵심 플로우:

```
CometBFT → PrepareProposal → ProcessProposal → FinalizeBlock
↓ ↓ ↓ ↓
EVMEngine 페이로드 생성 페이로드 검증 페이로드 실행
```

진입점: `halo/main.go` → `halocmd.New()`

2. 크로스체인 라이브러리 (lib/xchain)

위치: `lib/xchain/`

역할: 크로스체인 통신의 핵심 타입과 인터페이스 정의

주요 구성요소:

- `types.go` : 크로스체인 메시지, 블록 헤더, 증명 타입
- `provider.go` : 크로스체인 데이터 제공자 인터페이스
- `merkle.go` : 머클 트리 증명 시스템

핵심 타입:

- `Msg` : 크로스체인 메시지
- `BlockHeader` : 블록 헤더 정보
- `Attestation` : 크로스체인 증명
- `ConfLevel` : 확인 레벨 (Latest, Finalized, Min1, Min2)

3. Relayer (릴레이어)

위치: `relayer/`

역할: 크로스체인 메시지 전송 및 실행

주요 기능:

- 소스 체인에서 메시지 감지
- 목적지 체인으로 메시지 전송
- 실행 결과 추적 및 보고

진입점: `relayer/main.go` → `appcmd.New()`

4. Solver (솔버넷)

위치: `solver/`

역할: SolverNet 참조 구현

주요 기능:

- 크로스체인 거래 최적화
- 유동성 관리
- 수수료 계산 및 분배

진입점: `solver/main.go` → `solvercmd.New()`

5. Monitor (모니터링)

위치: `monitor/`

역할: 네트워크 상태 모니터링 및 메트릭 수집

주요 기능:

- 검증자 상태 추적
- 크로스체인 메시지 통계
- 성능 메트릭 수집
- 알림 및 경고 시스템

진입점: `monitor/main.go` → `appcmd.New()`

6. Contracts (스마트 컨트랙트)

위치: `contracts/`

역할: 온체인 로직 구현

구성요소:

- `core/` : 핵심 프로토콜 컨트랙트
- `avs/` : Eigen AVS 컨트랙트

- `bindings/` : Go 바인딩
- `allocs/` : 사전 배포 할당

7. Octane (EVM 모듈)

위치: `octane/`

역할: EVM용 Cosmos SDK 모듈 프레임워크

주요 기능:

- Engine API 구현 EVM 실행 클라이언트 지원
- ABCI 2.0 구현 합의 클라이언트 지원
- 커스텀 프리디플로이 지원

8. CLI (명령줄 인터페이스)

위치: `cli/`

역할: 운영자 및 개발자를 위한 도구

주요 기능:

- 네트워크 관리 명령
- 키 관리
- 트랜잭션 생성 및 전송

9. SDK (개발자 도구)

위치: `sdk/`

역할: Omni 애플리케이션 개발 지원

주요 기능:

- React 훅 제공

- SolverNet 인터페이스
- 개발자 도구 및 예제

10. E2E 테스트 프레임워크

위치: `e2e/`

역할: 엔드투엔드 테스트 및 네트워크 관리

주요 기능:

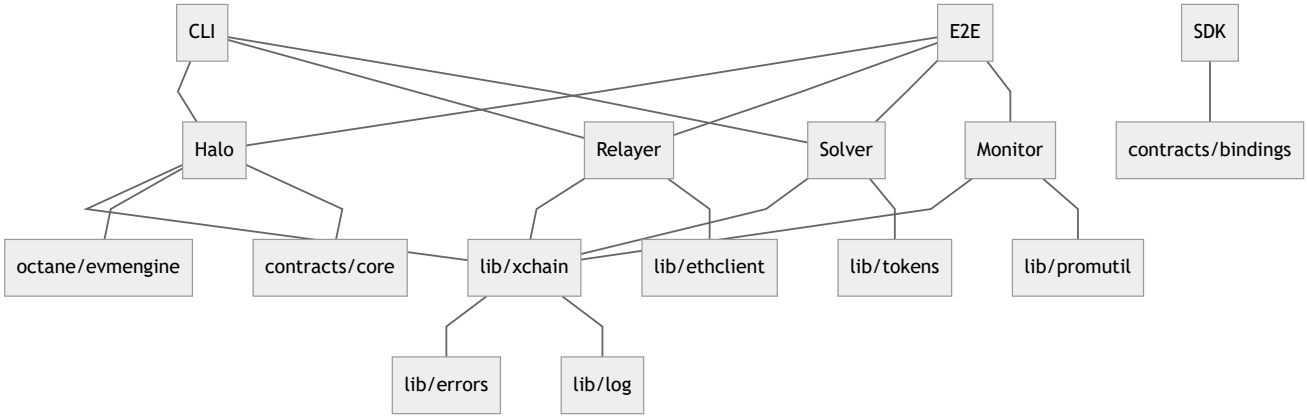
- Docker Compose 기반 테스트넷
- 매니페스트 기반 네트워크 구성
- 자동화된 테스트 실행

테스트 단계:

1. `setup` : 설정 파일 생성
2. `start` : Docker 컨테이너 시작
3. `wait` : 블록 생성 대기
4. `test` : 테스트 실행
5. `stop` : 컨테이너 중지
6. `cleanup` : 정리

컴포넌트 간 의존성 관계

의존성 매핑



데이터 플로우

1. **메시지 생성**: 소스 체인에서 크로스체인 메시지 발생
2. **합의 처리**: Halo가 메시지를 검증하고 합의 달성
3. **릴레이**: Relayer가 메시지를 목적지 체인으로 전송
4. **실행**: 목적지 체인에서 메시지 실행 및 결과 반환
5. **모니터링**: Monitor가 전체 과정을 추적하고 메트릭 수집

빌드 및 배포 시스템

Makefile 주요 타겟

- `build-docker` : Docker 이미지 빌드
- `devnet-deploy` : 개발 네트워크 배포
- `e2e-run` : E2E 테스트 실행
- `contract-bindings` : Go 컨트랙트 바인딩 생성

- `install-cli` : CLI 도구 설치

GoReleaser 설정

- `.goreleaser-snapshot.yaml` : 스냅샷 릴리스 설정
- `.goreleaser-official.yaml` : 공식 릴리스 설정
- 멀티 아키텍처 지원 (amd64, arm64)

Go 모듈 구조

주요 의존성

- **Cosmos SDK**: `github.com/cosmos/cosmos-sdk v0.50.14`
- **CometBFT**: `github.com/cometbft/cometbft v0.38.17`
- **Ethereum**: `github.com/ethereum/go-ethereum v1.15.7`
- **Solana**: `github.com/gagliardetto/solana-go v1.13.0`

내부 모듈 구조

모든 컴포넌트는 `github.com/omni-network/omni` 모듈 하위에 구성되어 있으며, 상대 경로를 통해 서로 참조합니다.

개발 환경 설정

필수 도구

- Go 1.24.0+
- Docker & Docker Compose
- GoReleaser

- Foundry (Solidity 개발)
- Node.js & pnpm (SDK 개발)

초기 설정

```
# 개발 환경 초기화
make setup

# Docker 이미지 빌드
make build-docker

# 개발 네트워크 시작
make devnet-deploy

# E2E 테스트 실행
MANIFEST=devnet1 make e2e-run
```

보안 및 품질 관리

코드 품질

- **Pre-commit 훅**: `.pre-commit-config.yaml`
- **Linting**: golangci-lint, solhint
- **테스트**: 단위 테스트, 통합 테스트, E2E 테스트

보안

- **Bug Bounty**: Immunefi 프로그램
- **코드 서명**: GPG 서명 필수
- **의존성 관리**: Go modules, 정기적 업데이트

네트워크 구성

환경별 구성

- **Devnet**: 로컬 개발 환경
- **Testnet**: 테스트 환경
- **Mainnet**: 프로덕션 환경

매니페스트 기반 구성

E2E 테스트 프레임워크는 TOML 매니페스트 파일을 통해 네트워크 토폴로지를 정의합니다.

결론

Omni 네트워크는 모듈화된 아키텍처를 통해 크로스체인 메시징, EVM 통합, 합의 메커니즘을 효과적으로 구현합니다. 각 컴포넌트는 명확한 역할과 책임을 가지며, 표준화된 인터페이스를 통해 상호작용합니다. 이러한 구조는 확장성, 유지보수성, 테스트 가능성을 보장합니다.